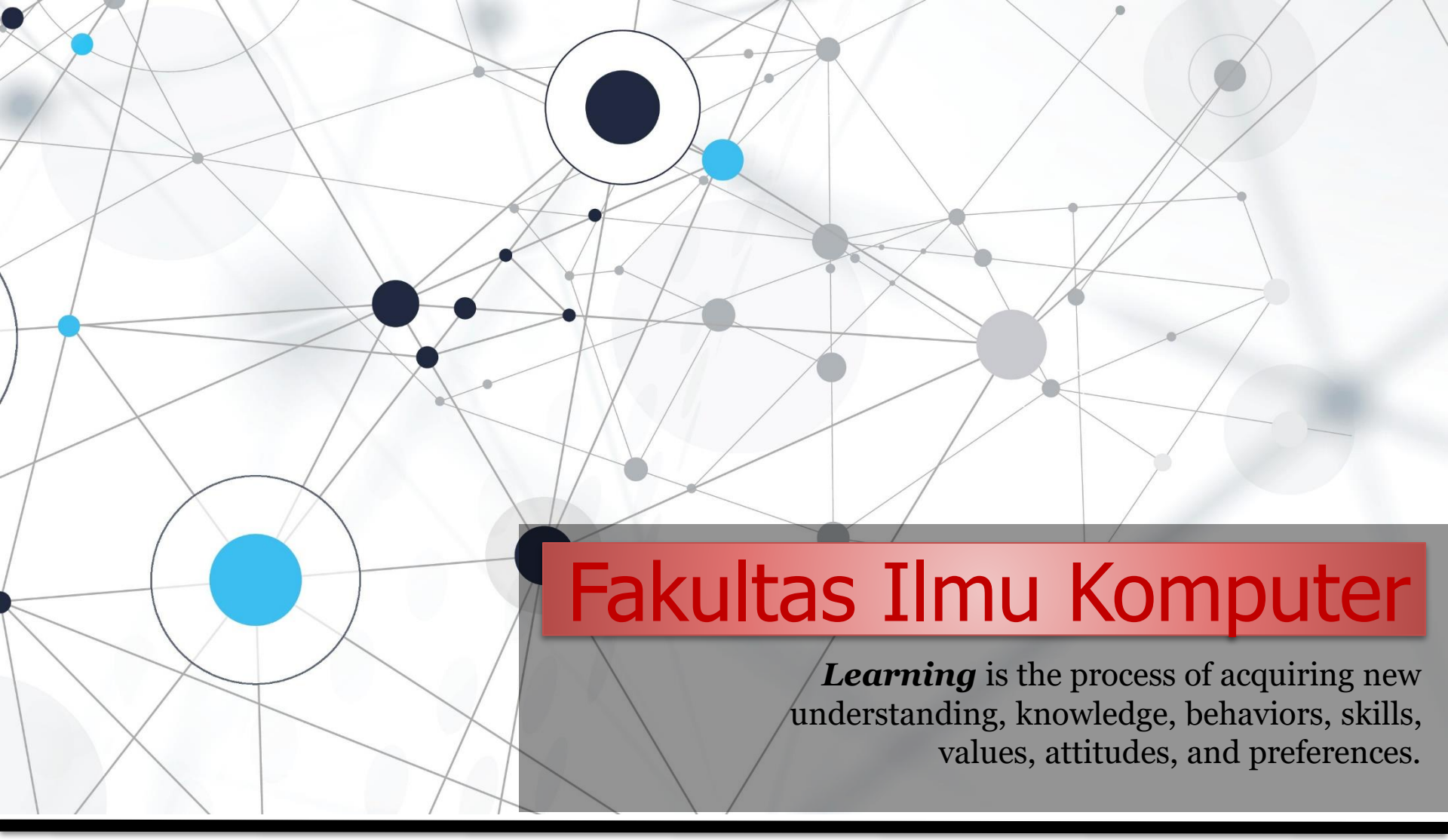


PHP Classes & Objects

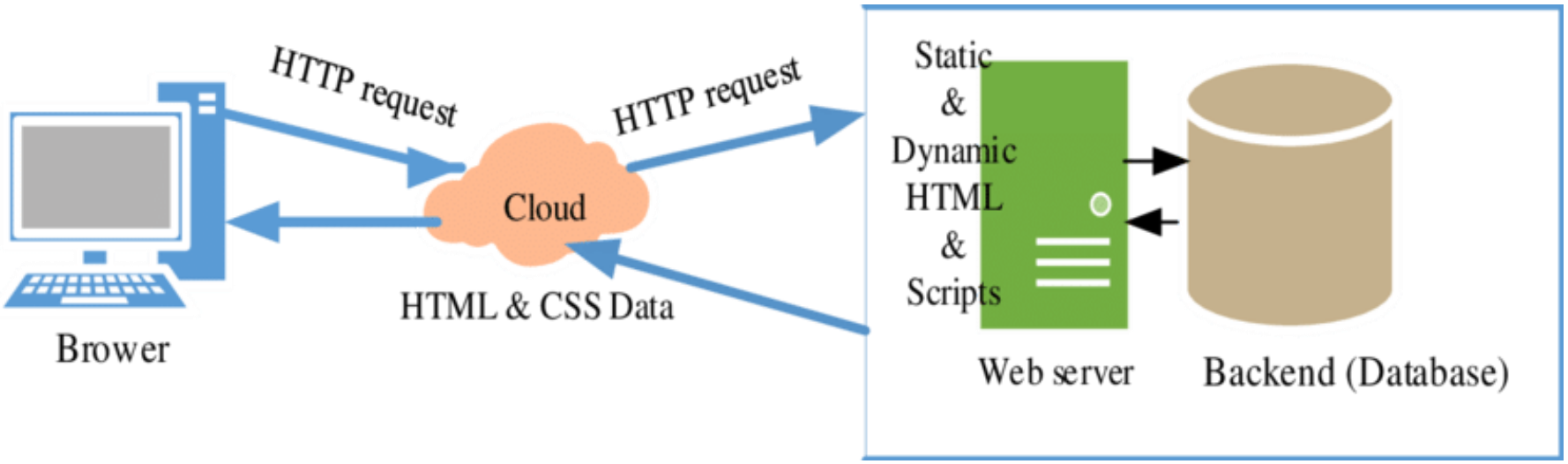
[PHP Programming]

Object Oriented Programming *in* PHP

Object Oriented Concepts


OOP
OBJECT
ORIENTED
PROGRAMMING

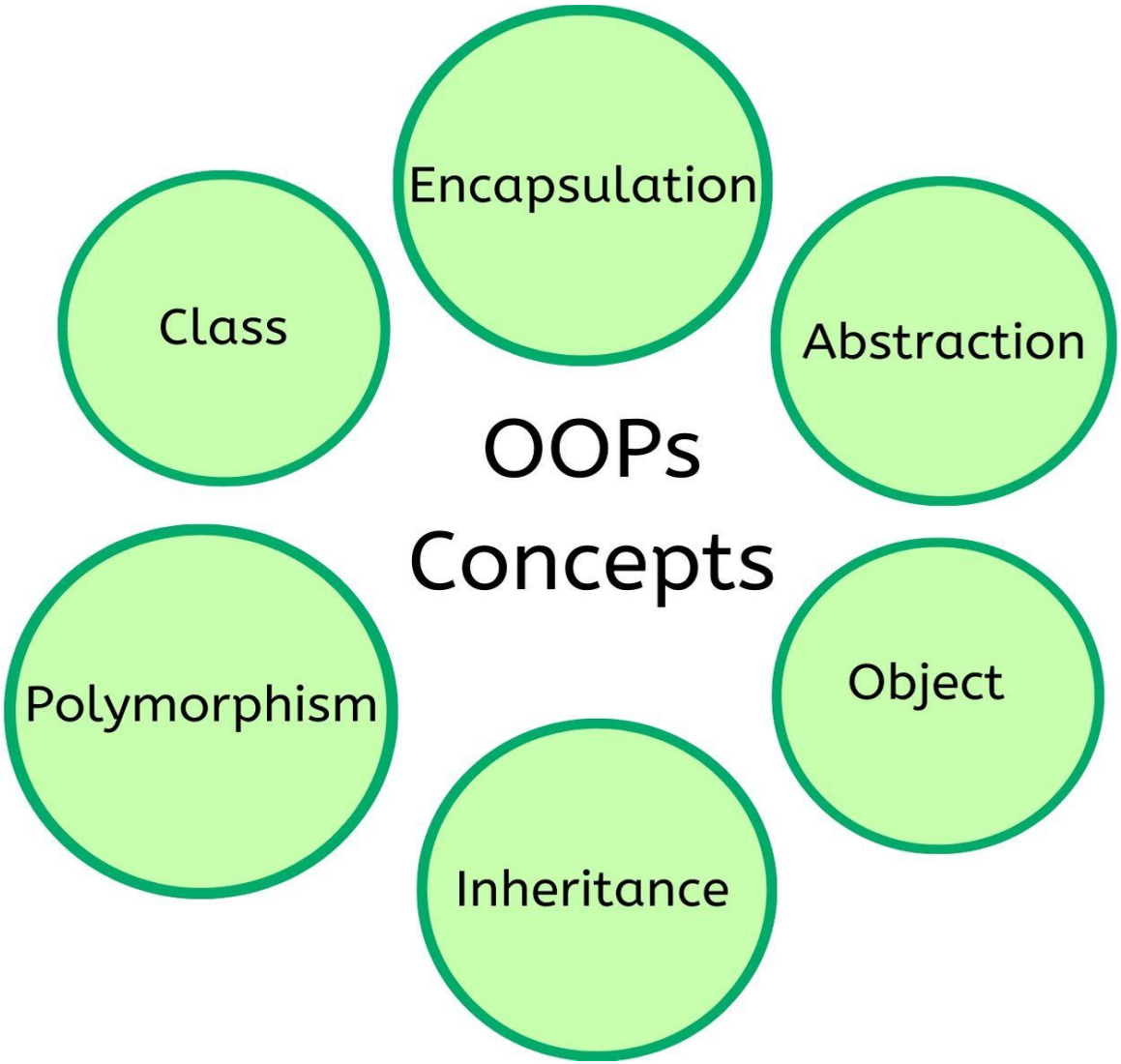
Why are *classes* and *objects* so important in web development, especially in back-end web development?



Business logic is the core of the application's functionality and ensures that the web/application works according to the needs and rules set by the business or project. In web development, separating the **business logic** from the **UI** and **user interaction** helps in keeping the code more structured, manageable, and well testable.

Types of PHP MVC framework

Framework	Description
 CodeIgniter https://codeigniter.com/	It is one of the most popular PHP MVC frameworks. It's lightweight and has a short learning curve. It has a rich set of libraries that help build websites and applications rapidly. Users with limited knowledge of OOP programming can also use it. CodeIgniter powered applications include; • https://www.pyrocms.com/ • http://www.shopigniter.com/
 Kohana http://kohanaframework.org	It's a Hierarchical Model View Controller HMVC secure and lightweight framework. It has a rich set of components for developing applications rapidly. Companies that use Kohana include; • https://go.wepay.com/ • https://kids.nationalgeographic.com/ • https://www.sittercity.com/
 CakePHP www.cakephp.org	It is modeled after Ruby on rails. It's known for concepts such as software design patterns, convention over configuration, ActiveRecord etc. CakePHP powered applications include; • http://invoicemachine.com/ • http://www.fmylife.com/
 www.framework.zend.com Zend	It is a powerful framework that is; • Secure, reliable, fast, and scalable • Supports Web 2.0 and creation of web services. It features APIs from vendors like Amazon, Google, Flickr, Yahoo etc. It's ideal for developing business applications. Zend powered applications include; • Pimcore CMS, • DotKernel.



PHP Classes & Object Orientation

OOP

Object Oriented Programming

What is Object Oriented Programming?

Object Oriented Programming

Object-Oriented Programming (OOP) is a ***programming model*** that is based on the ***concept*** of ***classes*** and ***objects***.

As opposed to **procedural programming** where the focus is on ***writing procedures or functions*** that perform operations on the ***data***, in ***object-oriented programming*** *the focus is on the creations of objects which contain* ***both data*** *and* ***functions together***.

Several Advantages *of* OOP

- ✓ It provides a ***clear modular structure*** for the programs.

- ✓ It helps you adhere to the "*don't repeat yourself*" (DRY) principle, and thus make your code ***much easier to maintain, modify and debug***.

- ✓ It makes it possible to create *more complicated behavior with less code* and *shorter development time* and *high degree of reusability (existing assets)*.

DRY Principle

The idea behind
Don't Repeat Yourself (DRY) principle
is reducing the repetition of code by abstracting out the code that are common for the application and placing them at a single place and ***reuse them instead of repeating*** it.

Understand Classes & Objects

- Classes and objects are the two main aspects of object-oriented programming. A class is a self-contained, independent collection of variables and functions which work together to perform one or more specific tasks, while objects are individual instances of a class.
- A ***class acts as a template or blueprint*** from which lots of individual objects can be created. *When individual objects are created, they inherit the same generic properties and behaviors, although each object may have different values for certain properties*.

Blueprint

Houses built according to the blueprint

Create a PHP File

- A class can be declared using the ***class keyword***, followed by the name of the class and a pair of curly braces ({}), as shown in the following example.
- The ***public keyword in*** properties (variable) and methods in the example, is an ***access modifier***, which indicates that this property or method is accessible from anywhere.
- **Properties or variables**

```php
<?php
class Rectangle
{
    // Declare  properties
    public $length = 0;
    public $width = 0;

    // Method to get the perimeter
    public function getPerimeter(){
        return (2 * ($this->length + $this->width));
    }

    // Method to get the area
    public function getArea(){
        return ($this->length * $this->width);
    }
}
?>
```

PHP Access Modifiers:
https://www.studytonight.com/php/php-access-modifiers

Access Properties/Variables & Methods

file name: *Rectangle.php*

```php
<?php
class Rectangle
{
    // Declare  properties
    public $length = 0;
    public $width = 0;

    // Method to get the perimeter
    public function getPerimeter(){
        return (2 * ($this->length + $this->width));
    }

    // Method to get the area
    public function getArea(){
        return ($this->length * $this->width);
    }
}
?>
```

Once a class has been defined, objects can be created from the class with the new keyword. Class methods and properties can directly be accessed through this object instance.

file name: *test.php*

```php
<?php
// Include class definition
require "Rectangle.php";

// Create a new object from Rectangle class
$obj = new Rectangle;

// Get the object properties values
echo $obj->length; // 0utput: 0
echo $obj->width; // 0utput: 0

// Set object properties values
$obj->length = 30;
$obj->width = 20;

// Read the object properties values again to show the change
echo $obj->length; // 0utput: 30
echo $obj->width; // 0utput: 20


// Call the object methods
echo $obj->getPerimeter(); // 0utput: 100
echo $obj->getArea(); // Output: 600
?>
```

1) The ***arrow symbol (->)*** is an OOP construct that is used to access contained properties and methods of a given object.
2) The ***pseudo-variable $this*** provides a reference to the calling object *i.e.* the object to which the method belongs.

Using Constructors & Destructors

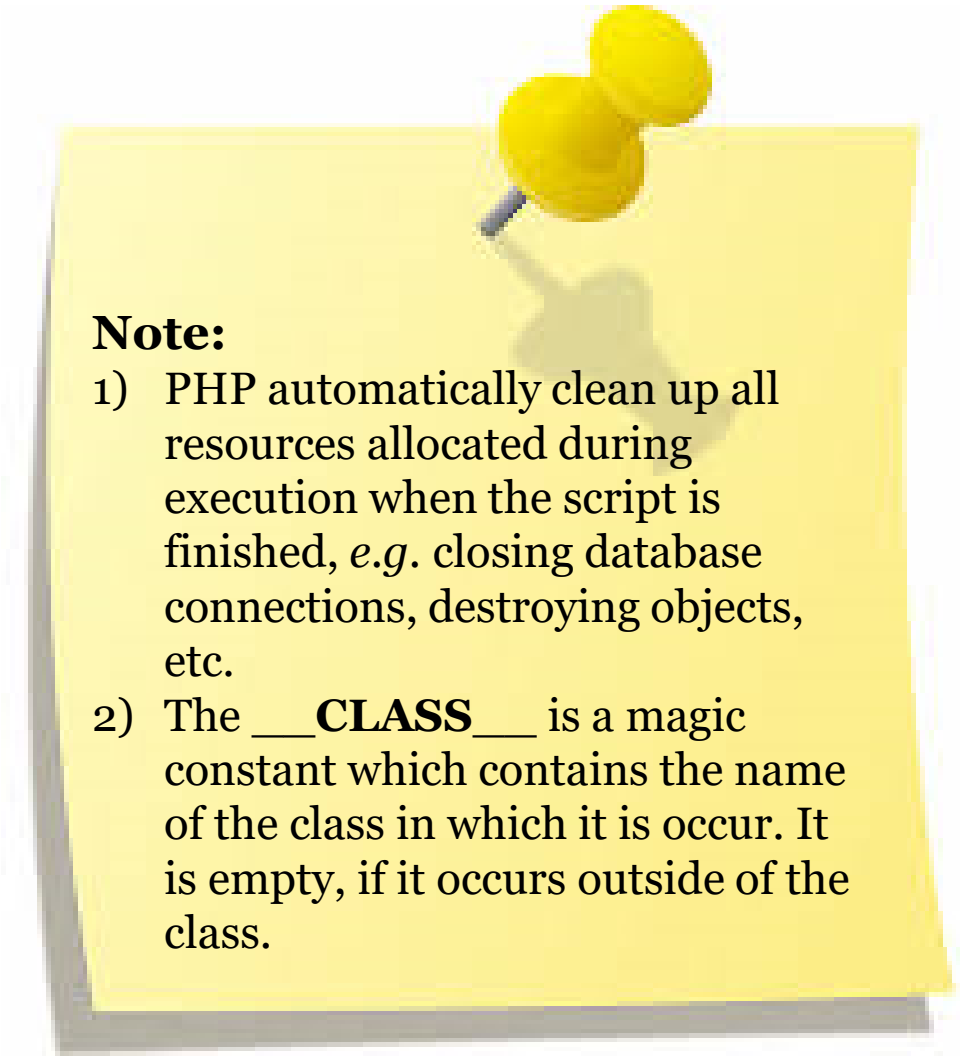
```php
<?php
class MyClass
{
    // Constructor
    public function __construct(){
        echo 'The class "' . __CLASS__ . '" was initiated!<br>';
    }

    // Destructor
    public function __destruct(){
    echo 'The class "' . __CLASS__ . '" was destroyed.<br>';
    }
}

// Create a new object
$obj = new MyClass;

// Destroy the object
unset($obj);

// Output a message at the end of the file
echo "The end of the file is reached.";
?>
```

PHP code will produce the following output:

The class "MyClass" was initiated!
The class "MyClass" was destroyed.
The end of the file is reached.

- PHP provides some magic methods that are executed automatically when certain actions occur within an object.
- The magic method ___construct() (known as *constructor*) is executed automatically whenever a new object is created.
- Similarly, the magic method ___destruct() (known as *destructor*) is executed automatically when the object is destroyed.
- A destructor function cleans up any resources allocated to an object once the object is destroyed.
- to explicitly trigger the destructor, you can destroy the object using the **PHP unset() function**.

Using Constructors & Destructors



Note:

1) PHP automatically clean up all resources allocated during execution when the script is finished, *e.g*. closing database connections, destroying objects, etc.
2) The **__CLASS__** is a magic constant which contains the name of the class in which it is occur. It is empty, if it occurs outside of the class.

Extend Classes through Inheritance

```php
<?php
// Include class definition
require "Rectangle.php";

// Define a new class based on an existing class
class Square extends Rectangle
{
    // Method to test if the rectangle is also a square
    public function isSquare(){
        if($this->length == $this->width){
            return true; // Square
        } else{
            return false; // Not a square
        }
    }
}

// Create a new object from Square class
$obj = new Square;

// Set object properties values
$obj->length = 20;
$obj->width = 20;

// Call the object methods
if($obj->isSquare()){
    echo "The area of the square is ";
} else{
    echo "The area of the rectangle is ";
};
echo $obj->getArea();
?>
```

- Classes can inherit the properties and methods of another class using the extends keyword.
- This process of extensibility is called inheritance.
- It is probably the most powerful reason behind using the ***object-oriented programming model***.
- The method **getArea()** already inherited from the parent **Rectangle class.**

Controlling *the* Visibility

Controlling the Visibility of Properties and Methods. There are three visibility keywords (from most visible to least visible): public, protected, private, which determines how and from where ***properties*** (*variables*) and ***methods*** **can be accessed and modified**.

- ✓ **public** — A public property or method can be accessed anywhere, from within the class and outside. This is the default visibility for all class members in PHP.

- ✓ **protected** — A protected property or method can only be accessed from within the class itself or in ***child*** or ***inherited*** classes i.e. classes that extends that class.

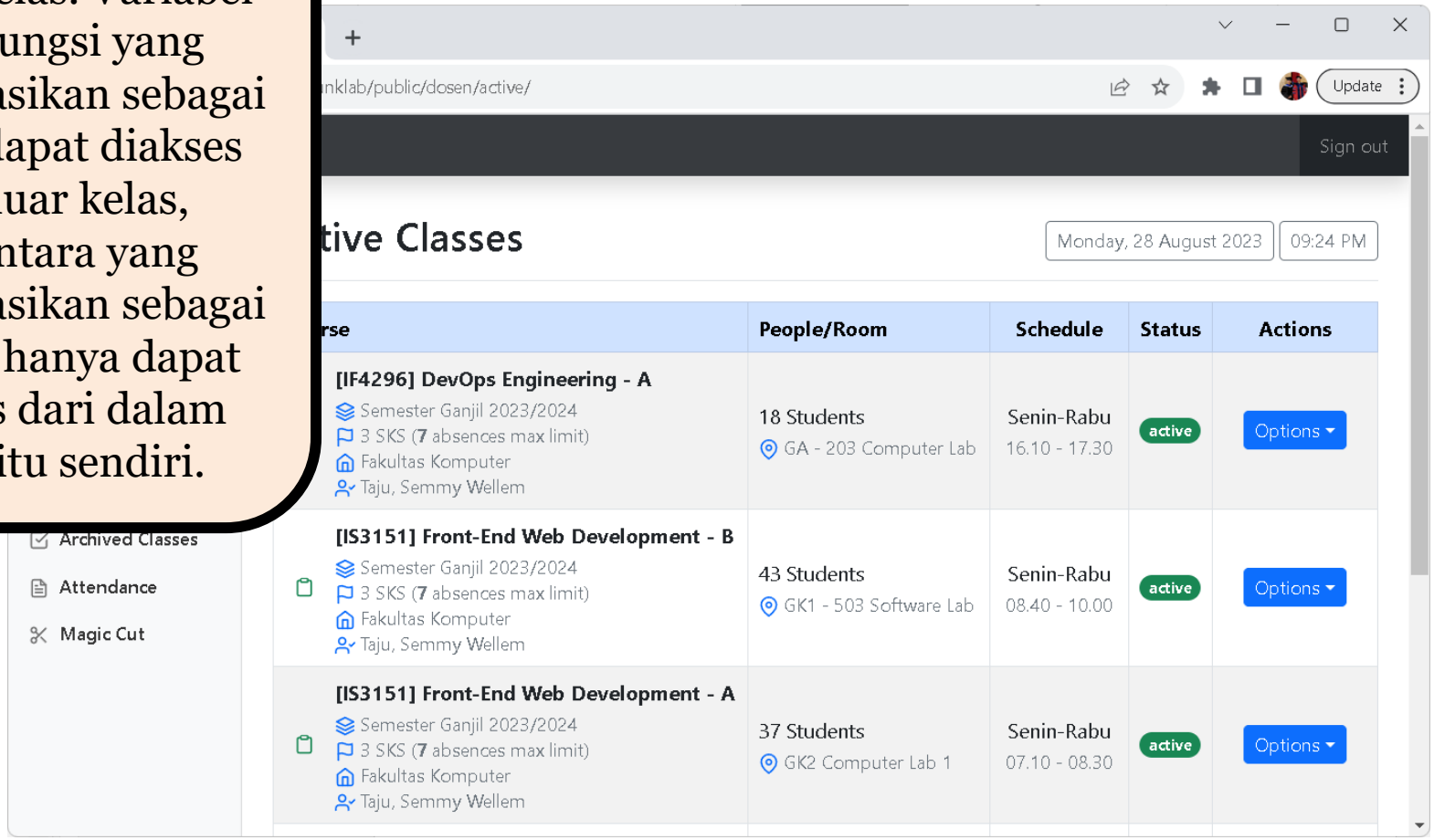
- ✓ **private** — A private property or method is accessible only from within the class that defines it. Even child or inherited classes cannot access private properties or methods.

Why do public and private variables/functions affect the security of a website, in this case for back-end web development?

Access Control (Pengendalian Akses) *Public* dan *private* mengontrol siapa yang memiliki akses ke variabel dan fungsi di dalam kelas. Variabel atau fungsi yang dideklarasikan sebagai *public* dapat diakses dari luar kelas, sementara yang dideklarasikan sebagai *private* hanya dapat diakses dari dalam kelas itu sendiri.

Uniform Resource Locator (URL):
https://presensi.test/unklab/public/**dosen**/**active**/

Perhatikan pada URL di atas, *text* yang diwarnai dengan warna merah dan biru artinya apa?



Perhatikan code di bawah ini !!!

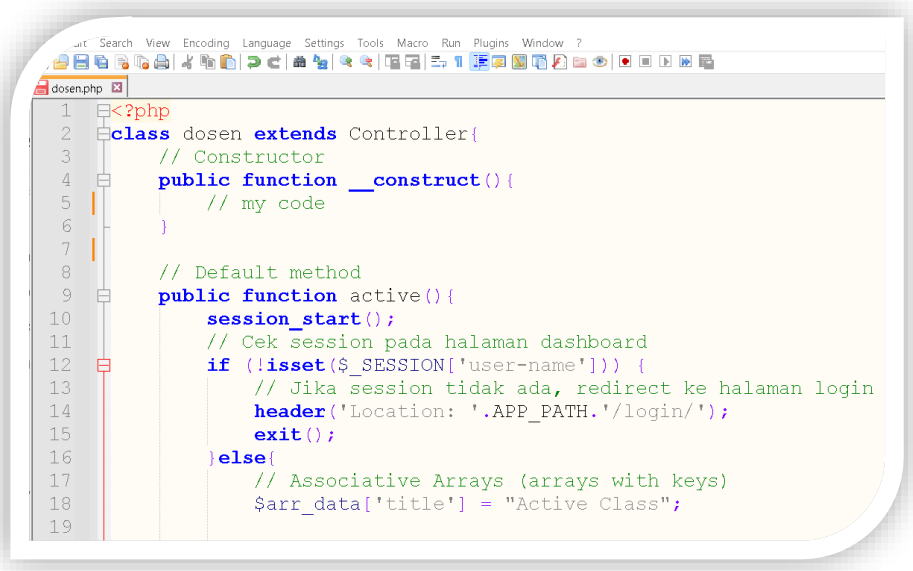
Uniform Resource Locator (URL):

https://presensi.test/unklab/public/**dosen**/**active**/

Ini adalah sebuah class.

Ini adalah sebuah fungsi.

Apa yang terjadi jika "public" access pada fungsi di samping menjadi "private" access?



Properties & Methods Visibility

```php
<?php
// Class definition
class Automobile
{
    // Declare  properties
    public $fuel;
    protected $engine;
    private $transmission;
}
class Car extends Automobile
{
    // Constructor
    public function __construct(){
        echo 'The class "' . __CLASS__ . '" was initiated!<br>';
    }
}

// Create an object from Automobile class
$automobile = new Automobile;

// Attempt to set $automobile object properties
$automobile->fuel = 'Petrol'; // ok
$automobile->engine = '1500 cc'; // fatal error
$automobile->transmission = 'Manual'; // fatal error

// Create an object from Car class
$car = new Car;

// Attempt to set $car object properties
$car->fuel = 'Diesel'; // ok
$car->engine = '2200 cc'; // fatal error
$car->transmission = 'Automatic'; // undefined
?>
```

When working with classes, you can even restrict access to its properties and methods using the visibility keywords for greater control.

How to access private variables?

```php
<?php
    class Person {
        // first name of person
        private $name;

        // public function to set value for name (setter method)
        public function setName($name) {
            $this->name = $name;
        }

        // public function to get value of name (getter method)
        public function getName() {
            return $this->name;
        }
    }

    // creating class object
    $john = new Person();

    // calling the public function to set fname
    $john->setName("John Wick");

    // getting the value of the name variable
    echo "My name is " . $john->getName();

?>
```

We can easily access the ***private variables of a class*** by defining **public functions** in the class. We can create separate functions to set value to private variables and to get their values too. **These functions are called Getters and Setters**.

Static Properties & Methods

```php
<?php
// Class definition
class HelloClass
{
    // Declare a static property
    public static $greeting = "Hello World!";

    // Declare a static method
    public static function sayHello(){
        echo self::$greeting;
    }
}
// Attempt to access static property and method directly
echo HelloClass::$greeting; // Output: Hello World!
HelloClass::sayHello(); // Output: Hello World!

// Attempt to access static property and method via object
$hello = new HelloClass;
echo $hello->greeting; // Strict Warning
$hello->sayHello(); // Output: Hello World!
?>
```

In addition to the visibility, properties and methods can also be declared as **static**, which makes them **accessible without needing an instantiation** of the class.

Static properties and methods can be accessed using the ***scope resolution operator (::)***,

Example (*like this*):
ClassName::$property and ClassName::method().

Perbedaan Fungsi

include() dan require()

Fungsi *Include* dan *Require*

Fungsi include():
Dalam manual PHP website menyatakan bahwa "*The include statement includes and evaluates the specified file*". Artinya fungsi include() akan menyertakan dan mengevaluasi seluruh program yang ada di *file* yang disertakan. Jika terdapat *error* pada program yang disertakan, maka *error* akan ditampilkan di layar dan jika *file* yang disertakan ternyata tidak ditemukan (*mungkin karena lokasi yang salah atau memang file tidak ada*), maka program selanjutnya (*setelah include*) akan tetap dijalankan walaupun ditampilkan *error*.

Fungsi require():
Pada dasarnya sama dengan perintah *include*(). Perbedaannya hanya terletak pada saat *file* yang disertakan tidak ditemukan, maka perintah-perintah selanjutnya tidak akan dijalankan.

Read more (link):
https://achmatim.net/2013/10/20/php-perbedaan-fungsi-include-require-include_once-dan-require_once/

More about
Object Oriented Concepts

Reminder… a function

- Reusable piece of code.
- Has its own ‘local scope’.

```
function my_func($arg1,$arg2) {
    << function statements >>
}
```

Conceptually, what does a function represent?

…give the function something (arguments), it does something with them, and then returns a result…

Action or Method

What is a *class*?

Conceptually, a class represents an **object**, with associated methods and variables

Class Definition

```
<?php
class dog {
  public $name;
  public function bark() {
     echo 'Woof!';
  }
}
?>
```

An example class definition for a dog. The dog object has a single attribute, the name, and can perform the action of barking.

Class Definition

```
<?php
class dog {
  public $name;
  public function bark() {
     echo 'Woof!';
  }
}
?>
```

Define the **name** of the class.

Class Definition

```
<?php
class dog {
  public $name;
  public function bark() {
     echo 'Woof!';
  }
}
?>
```

Define an object attribute (variable), the dog's name.

Class Defini

```
<?php
class dog {
  public $name;
  public function bark() {
      echo 'Woof!';
  }
}
?>
```

Define an object action (function), the dog's bark.

Class Definition

```
<?php
class dog {
  public $name;
  public function bark() {
     echo 'Woof!';
  }
}
?>
```

End the class definition

Class Defintion

Similar to defining a function..

*The definition **does not do anything by itself**. It is a blueprint, or description, of an object. To do something, you need to **use** the class…*

Class Usage

```
<?php
require('dog.class.php');
$puppy = new dog();
$puppy->name = 'Rover';
echo "{$puppy->name} says ";
$puppy->bark();
?>
```

Class Usage

```
<?php
require('dog.class.php');
$puppy = new dog();
$puppy->name = 'Rover';
echo "{$puppy->name} says ";
$puppy->bark();
?>
```

Include the class definition

Class Usage

```
<?php
require('dog.class.php');
$puppy = new dog();
$puppy->name = 'Rover';
echo "{$puppy->name} says ";
$puppy->bark();
?>
```

Create a new **instance** of the class.

Class Usage

```
<?php
require('dog.class.php');
$puppy = new dog();
$puppy->name = 'Rover';
echo "{$puppy->name} says ";
$puppy->bark();
?>
```

Set the name variable **of this instance** to ‘Rover’.

Class Usage

```
<?php
require('dog.class.ph
$puppy = new dog();
$puppy->name = 'Rover';
echo "{$puppy->name} says ";
$puppy->bark();
?>
```

Use the name variable of this instance in an echo statement..

Class Usage

```
<?php
require('dog.class.php');
$puppy = new dog();
$puppy->name = 'Rover';
echo "{$puppy->name} says ";
$puppy->bark();
?>
```

Use the dog object bark method.

Class Usage

```
<?php
require('dog.class.php');
$puppy = new dog();
$puppy->name = 'Rover';
echo "{$puppy->name} says ";
$puppy->bark();
?>
```

[example file: classes1.php]

One dollar and one only…

```
$puppy->name = 'Rover';
```

The most common mistake is to use more than one dollar sign when accessing variables. The following means something entirely different..

~~`$puppy->$name = 'Rover';`~~

Using attributes within the class..

- If you need to use the class variables within any class actions, use the special variable **$this** in the definition:

```
class dog {
 public $name;
 public function bark() {
  echo $this->name.' says Woof!';
 }
}
```

Constructor methods

- A **constructor** method is a function that is automatically executed when the class is first instantiated.
- Create a constructor by including a function within the class definition with the **__construct name**.
- Remember.. if the constructor requires arguments, they must be passed when it is instantiated!

Constructor Example

```
<?php
class dog {
  public $name;
  public function __construct($nametext) {
     $this->name = $nametext;
  }
   public function bark() {
     echo 'Woof!';
  }
}
?>
```

Constructor function

Constructor Example

```
<?php
      …
$puppy = new dog('Rover');
      …
?>
```

Constructor arguments are passed during the instantiation of the object.

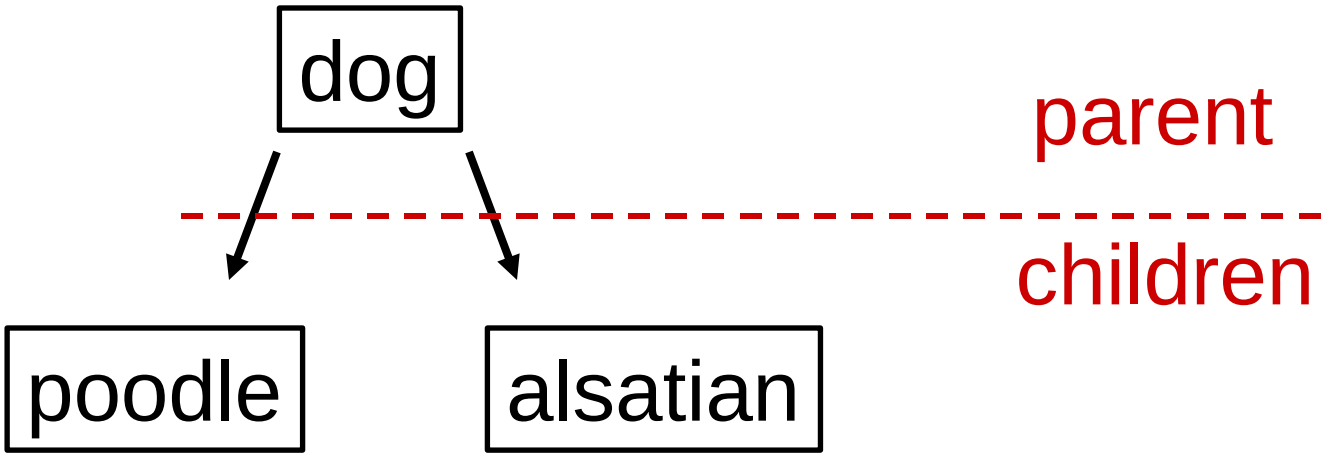
Class Scope

- Like functions, **each instantiated object** has its own local scope.

  e.g. if 2 different dog objects are instantiated, **`$puppy1`** and **`$puppy2`**, the two dog names **`$puppy1->name`** and **`$puppy2->name`** are entirely independent..

Inheritance

- The real power of using classes is the property of inheritance – creating a hierarchy of interlinked classes.



Inheritance

- The child classes ‘inherit’ all the methods and variables of the parent class, and can add extra ones of their own.

  e.g. the child classes poodle inherits the variable ‘name’ and method ‘bark’ from the dog class, and can add extra ones…

Inheritance example

The American Kennel Club (AKC) recognizes three sizes of poodle - Standard, Miniature, and Toy…

```
class poodle extends dog {
        public $type;
        public function set_type($height) {
                if ($height<10) {
                        $this->type = 'Toy';
                } elseif ($height>15) {
                        $this->type = 'Standard';
                } else {
                        $this->type = 'Miniature';
                }
        }
}
```

Inheritance example

The American Kennel Club (AKC) recognizes three sizes of poodle - Standard, Miniature, and Toy…

```
class poodle extends dog {
        public $type
        public fun
                if
                } el
                } el
                }
        }
}
```

Note the use of the **`extends`** keyword to indicate that the poodle class is a child of the dog class…

Inheritance example

```
…
$puppy = new poodle('Oscar');
$puppy->set_type(12); // 12 inches high!
echo "Poodle is called {$puppy->name}, ";
echo "of type {$puppy->type}, saying ";
echo $puppy->bark();
…
```

…a poodle will always ‘Yip!’

- It is possible to over-ride a parent method with a new method if it is given the same name in the child class..

```
class poodle extends dog {
        …
        public function bark() {
                echo ‘Yip!’;
        }
        …
}
```

Child Constructors?

- If the child class possesses a constructor function, it is executed **and any parent constructor is ignored**.
- If the child class does not have a constructor, the parent's constructor is executed.
- If the child and parent does not have a constructor, the grandparent constructor is attempted…
- … etc.

Objects within Objects

- It is perfectly possible to include objects within another object..

```
class dogtag {
    public $words;
}

class dog {
    public $name;
    public $tag;

    public function bark() {
        echo "Woof!\n";
    }
}
```

```
…
$puppy = new dog;
$puppy->name = “Rover";
$poppy->tag = new dogtag;
$poppy->tag->words = “blah”;
…
```

Deleting objects

- So far our objects have not been destroyed till the end of our scripts..
- Like variables, it is possible to explicitly destroy an object using the **`unset()`** function.

A copy, or not a copy..

- Entire objects can be passed as arguments to functions, and can use all methods/variables within the function.
- Remember however.. like functions the object is **COPIED** when passed as an argument unless you specify the argument as a reference variable `&$variable`

Why Object Orientate?

Reason 1

Once you have your head round the concept of objects, intuitively named object orientated code becomes **easy to understand**.

e.g.

```
$order->display_basket();
$user->card[2]->pay($order);
$order->display_status();
```

Why Object Orientate?

Reason 2

Existing code becomes easier to maintain.

e.g. If you want to extend the capability of a piece of code, you can merely edit the class definitions…

Why Object Orientate?

Reason 3

New code becomes much quicker to write once you have a suitable class library.

e.g. Need a new object..? Usually can extend an existing object. A lot of high quality code is distributed as classes (e.g. http://pear.php.net).

PHP4 vs. PHP5

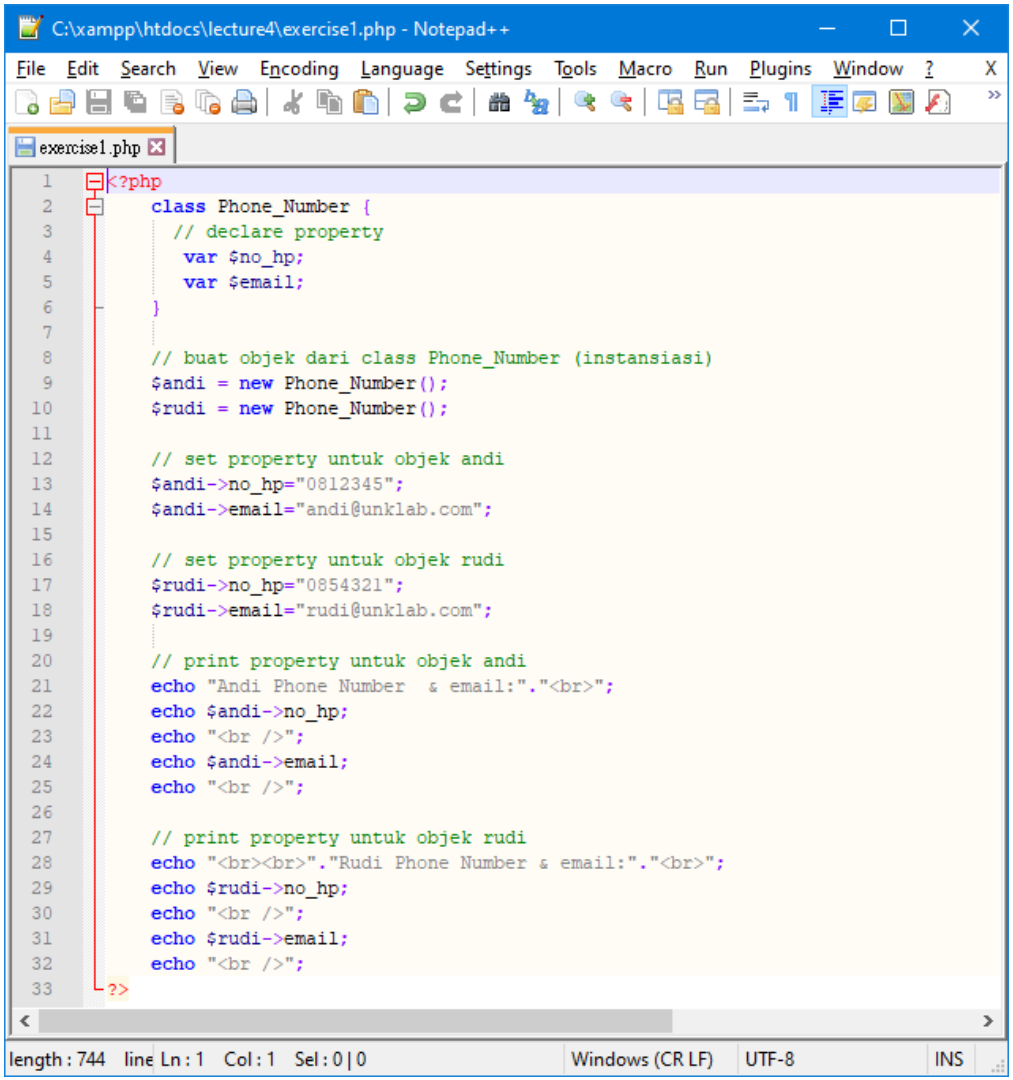
- OOP purists will tell you that the object support in PHP4 is sketchy. They are right, in that a lot of features are missing.
- PHP5 OOP system has had a big redesign and is **much** better.

…but it is worth it to produce OOP code in either PHP4 or PHP5…

Exercises *for* Students

Class, Object, Property & Method

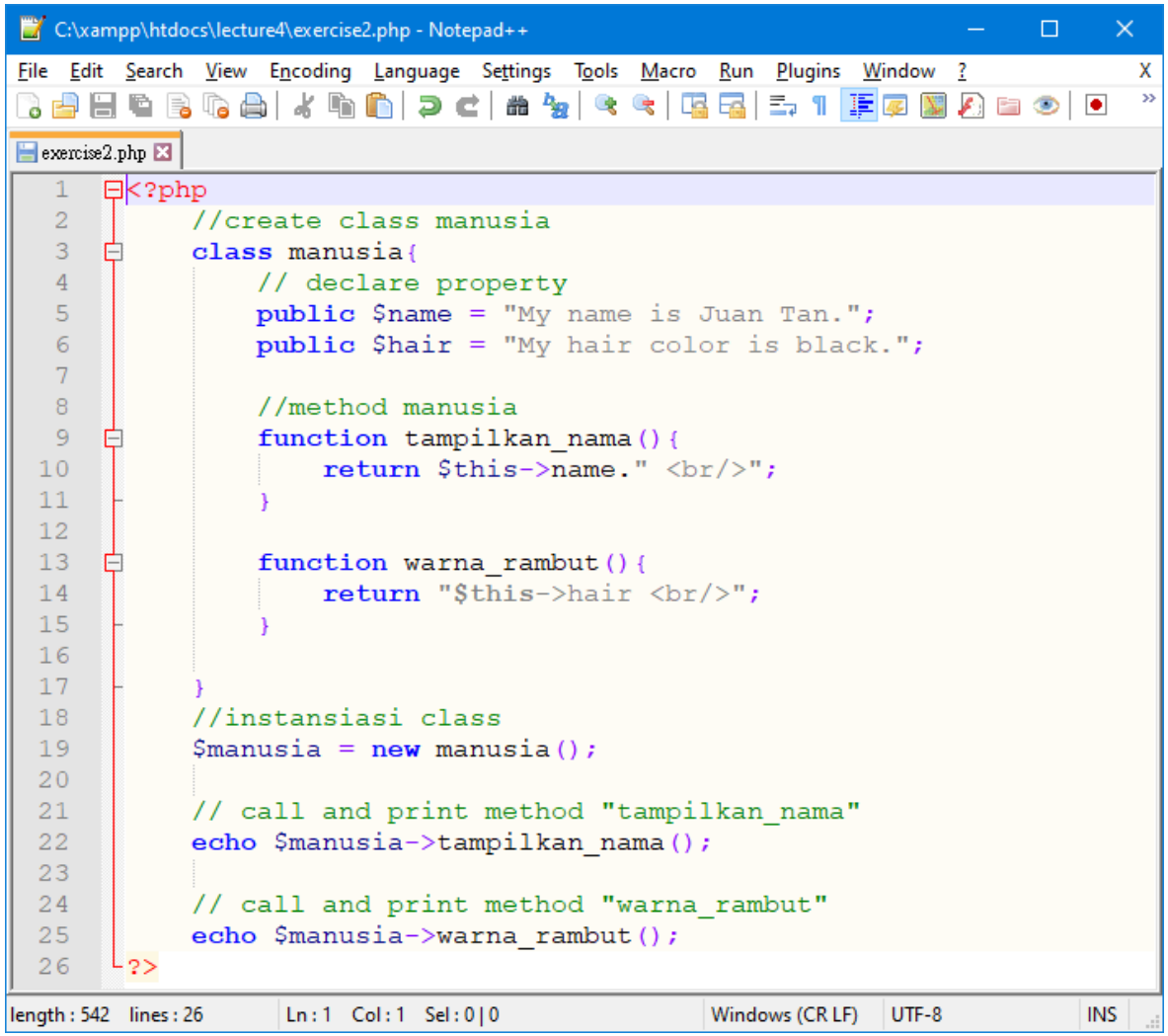
Create directory : C:\xampp\htdocs\lecture4\
Create file : exercise1.php



- The *var keyword* in PHP is used to declare a property or variable of class which is *public* by default.
- The *var keyword* is same as public when declaring variables or property of a class.

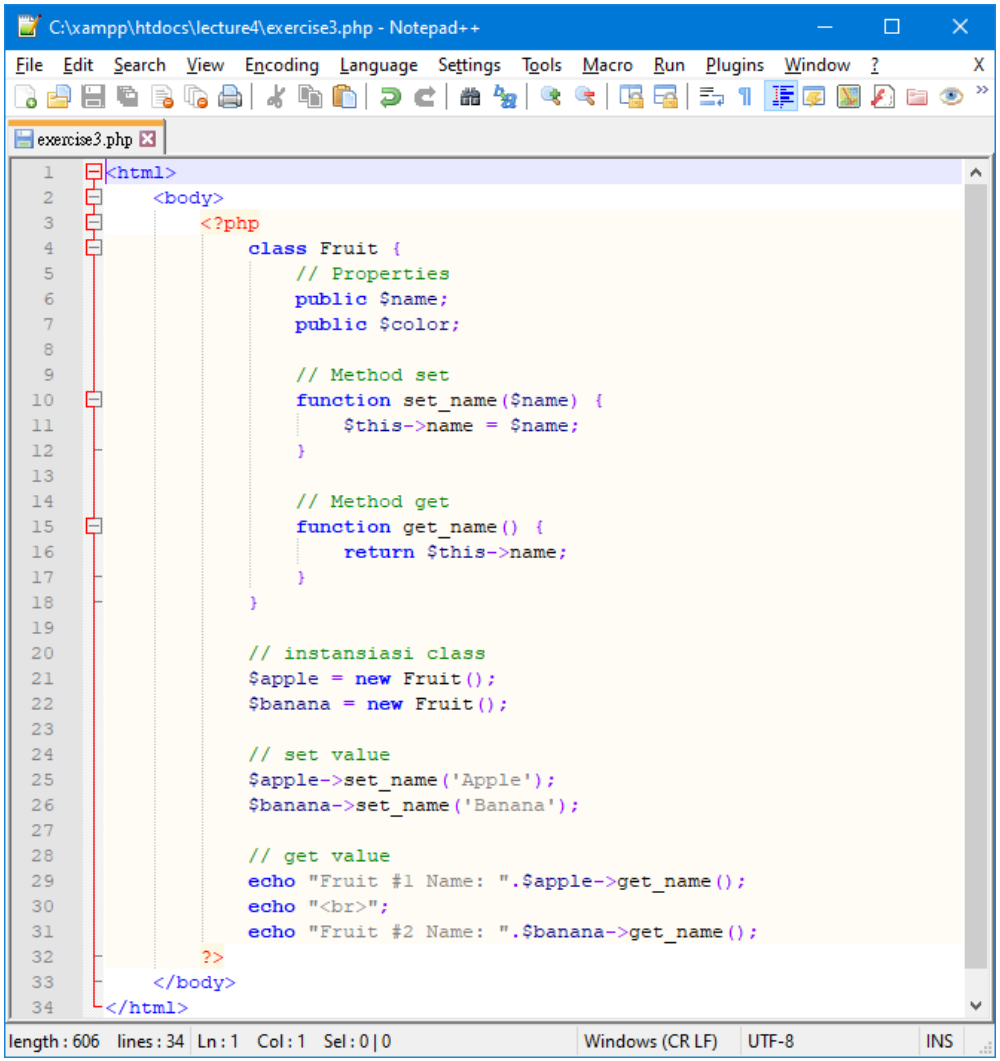
Class, Object, Property & Method

Create directory : C:\xampp\htdocs\lecture4\
Create file : exercise2.php



Class, Object, Property & Method

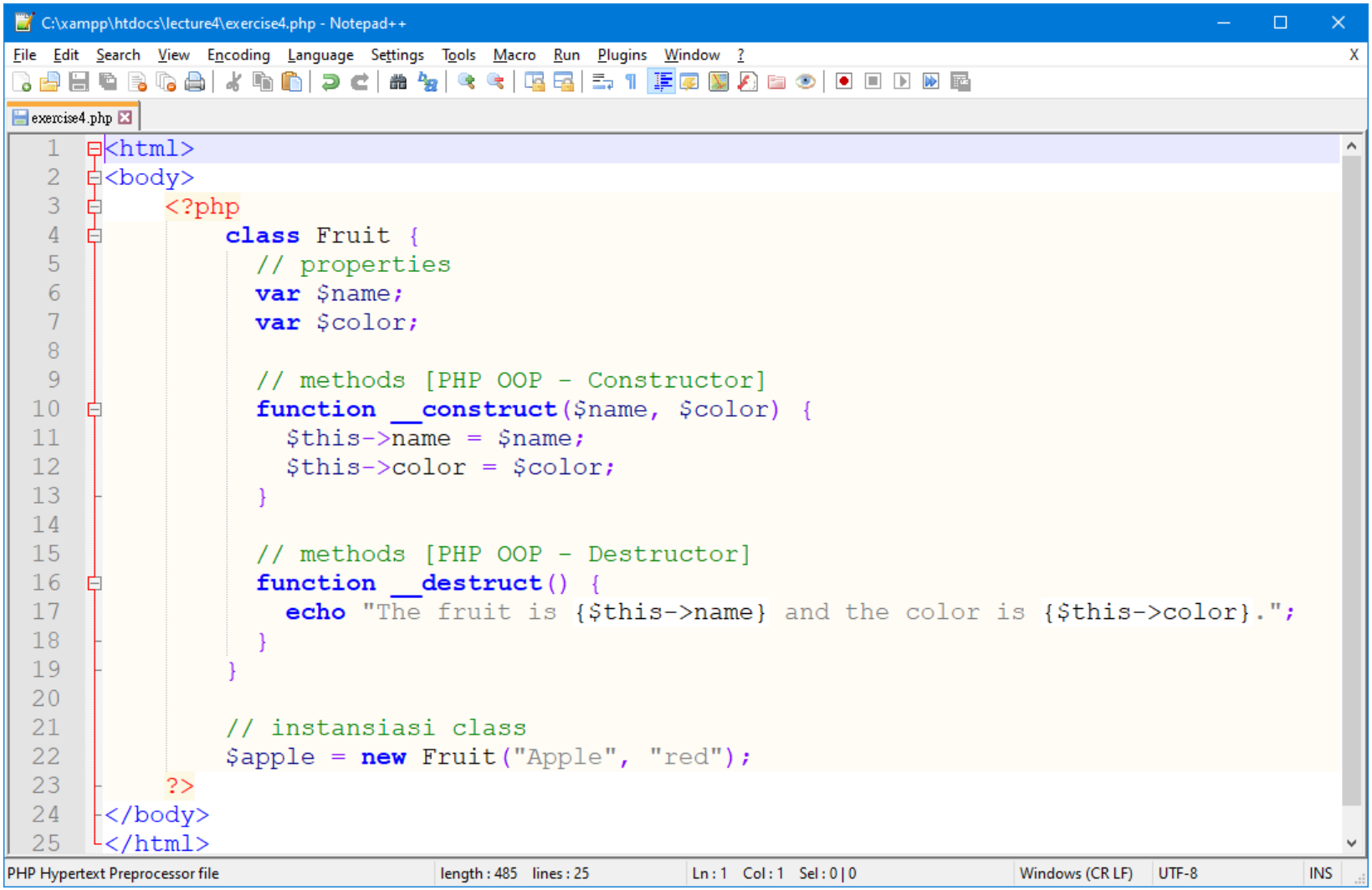
Create directory : C:\xampp\htdocs\lecture4\
Create file : exercise3.php




```php
<html>
    <body>
        <?php
            class Fruit {
                // Properties
                public $name;
                public $color;

                // Method set
                function set_name($name) {
                    $this->name = $name;
                }

                // Method get
                function get_name() {
                    return $this->name;
                }
            }

            // instansiasi class
            $apple = new Fruit();
            $banana = new Fruit();

            // set value
            $apple->set_name('Apple');
            $banana->set_name('Banana');

            // get value
            echo "Fruit #1 Name: ".$apple->get_name();
            echo "<br>";
            echo "Fruit #2 Name: ".$banana->get_name();
        ?>
    </body>
</html>
```

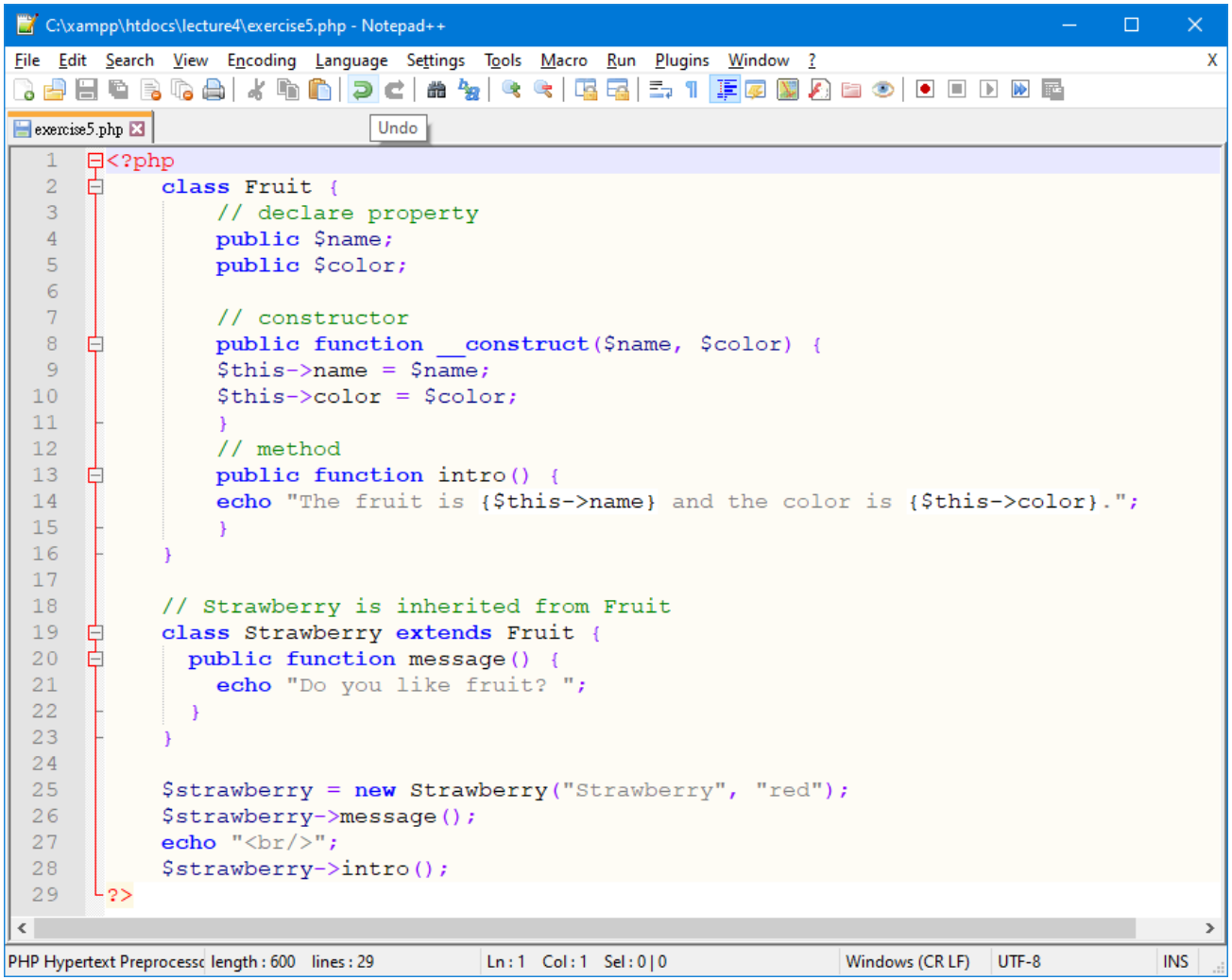
Class, Object, Property & Method

Create directory : C:\xampp\htdocs\lecture4\
Create file : exercise4.php



PHP OOP - Inheritance

Create directory : C:\xampp\htdocs\lecture4\
Create file : latihan5.php



Tambahkan 2 method (**get_fruit_name()** dan **get_fruit_color()**) pada class Latihan 4 dan 5 dan tampilkan output di browser dengan menggunakan object.

Reference (#1)

Object Oriented Programming in PHP
https://www.tutorialspoint.com/php/php_object_oriented.htm

PHP OOP - Classes and Objects
https://www.w3schools.com/php/php_oop_classes_objects.asp

PHP Classes and Objects
https://www.tutorialrepublic.com/php-tutorial/php-classes-and-objects.php

Classes, objects, methods and properties
https://phpenthusiast.com/object-oriented-php-tutorials/create-classes-and-objects

Object-Oriented PHP With Classes and Objects
https://code.tutsplus.com/tutorials/basics-of-object-oriented-programming-in-php--cms-31910

Class and Object in PHP
https://linuxhint.com/class_objects_php/

PHP Classes and Objects
https://arieljakubowski.medium.com/php-classes-and-objects-bde14a99139a

PHP Access Modifiers
https://www.studytonight.com/php/php-access-modifiers

PHP: Perbedaan Fungsi include(), require(), include_once() dan require_once()
https://achmatim.net/2013/10/20/php-perbedaan-fungsi-include-require-include_once-dan-require_once/

END PRESENTATION

Thank you for your attention

Instructor: S – W – T